

A New Algorithmic Approach to Finding Minimum Spanning Tree

Afsana Khan¹, Afrida Anzum Aesha², Juthi Sarker³

Department of Computer Science and Engineering

Military Institute of Science and Technology, Dhaka, Bangladesh

afsana29khan@gmail.com¹, afrida660@gmail.com² and tanudas71.21@gmail.com³

Abstract— Spanning tree of a graph is formed when each and every vertex of a graph are connected having no cycles in them and therefore minimum spanning tree as its name refers, is the tree with the smallest possible length among all spanning trees. Calculating minimum spanning tree of a graph has always been a common problem throughout ages. A number of efficient algorithms has been already developed for this problem. In this paper a different approach has been proposed where we profusely used sets and disjoint sets union data structure for reducing the number of edges under consideration while determining minimum spanning tree of a graph.

Index Terms—Minimum spanning tree, Sets and disjoint algorithm, Subsets, Complexity, Nearest vertex

I. INTRODUCTION

Minimum spanning tree problem is one of the most well-studied problems in the field of graph algorithms and is also a sub problem of NP hard problem like Travelling Salesman Problem [1]. Besides it has direct applications in the field of taxonomy, computer networks [2], medical imaging [3], water supply networks and transportation networks [4] etc.

Since many years a number of approaches have been made to find efficient solution for this problem. The long history of algorithms for MST problem can be found on an article [5] written by Graham and Held. There are four classic algorithms used till now in order to find MST of a connected graph; Ottawar Boruvka's algorithm which keeps on finding the cheapest edge from each component until only a single component is left. The algorithm operates in $O(|E|\log(|V|))$ time, here E and V are the number of edges and vertices of a connected graph respectively. The algorithm designed by Robert C.Prim [6], widely known as Prim's algorithm starts by using a vertex as a source and keeps on adding edges at a time until all the vertices becomes included in the tree. The algorithm makes use of efficient priority queue and its complexity depends on the data structure used to implement the priority queue. The time complexities are $O(|V|^2)$, $O(|E|\log(|E|))$ and $O(|E| + |V|\log(|V|))$ in case of implementing priority queue using adjacency matrix, binary heap & adjacency list and Fibonacci heap & adjacency list respectively. The algorithm developed by Joseph Kruskal [7] finds minimum spanning tree by sorting the edges in an increasing order and considering each vertex as a forest. It keeps on adding an edge at a time with a checking of no formation of cycles with the set and disjoint set union data structure. Thus, Kruskal's algorithm calculates the MST of a graph. Another algorithm completely opposite of Kruskal's algorithm widely

known as Reverse-Delete algorithm [8] sorts all the edges in decreasing order and keeps on deleting the edges. The deletion is done after checking whether the graph is disconnected or not, if the graph becomes disconnected upon deletion then that edge is not considered for deletion. Kruskal's algorithm is better in case of sparse graphs whereas Prim's algorithm runs faster on denser graphs.

Karger, Tarjan and Klein [9] have used random sampling to find minimum spanning tree. It makes their proposed algorithm in $O(m)$ bound. The algorithm applies Boruvka's steps and random-sampling steps at the same time. Both of this two steps reduces the edges of the graph to fixed constant. It runs in $O(m\alpha(m, n))$ time. It is an improvement of the algorithm proposed by Karger which has a time bound of $O(n\log n + m)$. In [10] MST was proposed by vertex associative cycle detection method. It checks if the highest degree vertex is a member of any cycle so that the most expensive edge can be removed. After eliminating all cycles, resultant MST is obtained. Fredman and Willard [11] have modeled an linear-time algorithm which allows bit manipulation of the binary representations of the edge weights. Osipov, Sanders and Singler [12] considered an algorithm for MST called *Filter-Kruskal*. They used quicksort for sorting edges and filtered the heavy edges that fall within the same component. The algorithm calls Kruskal's algorithm in the small input and for lighter part of the edges recursively. The running time of it is $O(m + n\log(n\log(m/n)))$.

For large dataset C. Zhong [13] proposed a fast minimum spanning tree algorithm based on K-means which uses a divide and conquer scheme in first stage and in second stage called refinement stage converts neighboring pair into a cluster with theoretical time complexity of $O(N^{1.5})$, which is faster than the conventional MST algorithms with $O(N^2)$. Saxena [14] also proposed a MST that focused on minimizing the time complexity using clustering. Divide method was used to divide the dataset and in conquer method local MST were created and combined to final MST for every cluster. Complexity of joining clusters was reduced to $2\sqrt{n}$ from $O(n)$. Lonvcar [15] implemented two parallel algorithms based on Prim's and Kruskal to find MST targeting message passing parallel machines with distributed memory. Bader [16] proposed four parallel MST with three variation of Boruvka algorithm using shared memory.

The algorithm proposed in this paper uses sets and

disjoint sets union data structure as like as in Kruskal's Algorithm but it does not need to perform sorting on the edges. All the edges are not brought under consideration while checking the formation of cycles as considerable number of edges are eliminated in the first step which will not belong to the minimum spanning tree. Therefore, the objective of this paper is to propose a minimum spanning tree algorithm for a connected weighted graph. The algorithm compares edge weights randomly and uses set disjoint method for finding MST.

The rest of the paper has been organized as: Section II presents working method of the proposed algorithm, Section IV describes the time complexity analysis of the algorithm, Section III and Section V discusses the simulation & proof of correctness of the algorithm. Finally Section VI concludes the paper.

II. PROPOSED ALGORITHM

A. Problem Statement

Let $G(V, E)$ be a connected graph where V and E are the total number of vertices and edges of the graph respectively. The goal is to find a tree that connects all the vertices V and total cost of the edges i.e. cost of the tree remains minimum.

B. Working Mechanism

1) **Phase-1: Formation of subsets:** The vertices, also termed as nodes will have a weight of its own known as node weight. The node weight of each node is initialized to infinity (Line 1-3). By traversing all the edges, one edge is selected at a time and its both end nodes are determined. If the node weights of the end nodes are greater than the weight of the edge between them then the node weights are updated with the edge weight. After traversing the entire edge list we see the node weights of each node is updated with the distance to the nearest node from that particular node (Line 4 – 13).

Traversing each node, we select the nearest vertex from it using its node weight. We obtained subsets of the graph where each node is connected to its nearest vertex i.e. connected to other node with the minimum edge weight (Line 14 – 16). It is to be noted that during the formation of subsets only those edges are used that are confirmed to belong to the MST, rest of the unused edges denoted by E' may contain only few edges that will be added to the MST. If S number of subsets are formed, then only $S - 1$ edges are required from the E' edges in order to connect all the subsets without cycle formation.

2) **Phase-2: Joining of subsets:** After the implementation till Line 16, a forest is formed or it can be said that subsets are formed. Each subset will have at least two nodes since we created the subset joining cheapest edge of nodes and thus a subset can be referred to as a supernode. Each and every supernode will now have a weight known as supernode weight. The supernode weights of all the supernodes are initialized to infinity (Line 17-19).

Now only the unused edges are traversed (Line 20 - 33) and at every iteration both end nodes of the edge are

Algorithm 1 Algorithm (G)

```

1: for each  $v \in G.V$  do
2:    $W_v(v) \leftarrow \infty$ 
3: end for
4: for each  $e \in G.E$  do
5:    $a \leftarrow \text{one end of } e$ 
6:    $b \leftarrow \text{other end of } e$ 
7:   if  $W_v(a) > W(e)$  then
8:      $W_v(a) \leftarrow W(e)$ 
9:   end if
10:  if  $W_v(b) > W(e)$  then
11:     $W_v(b) \leftarrow W(e)$ 
12:  end if
13: end for
14: for each  $v \in G.V$  do
15:   Select cheapest edge for  $v$  by which it connects
      another vertex
16: end for
17: for each  $s \in G.Subsets$  do
18:    $W_s(s) \leftarrow \infty$ 
19: end for
20: for each  $e \in G.E'$  do
21:    $S_1 \leftarrow \text{subset in which } p \text{ belongs}$ 
22:    $S_2 \leftarrow \text{subset in which } q \text{ belongs}$ 
23:    $p \leftarrow \text{one end of } e$ 
24:    $q \leftarrow \text{other end of } e$ 
25:   if  $\text{FindSet}(p) \neq \text{FindSet}(q)$  then
26:     if  $W_s(S_1) > W(e)$  then
27:        $W_s(S_1) \leftarrow W(e)$ 
28:     end if
29:     if  $W_s(S_2) > W(e)$  then
30:        $W_s(S_2) \leftarrow W(e)$ 
31:     end if
32:   end if
33: end for
34: for each  $S \in G \text{ subsets}$  do
35:   Select cheapest edge for  $S$  by which it connects
      another subset
36: end for

```

determined, it is checked whether both end nodes belong to the same supernode (subset) or not in order to stop formation of cycles. If both ends nodes of the edge belong to the same supernode then that edge is discarded as it will not belong to the MST. On the other side if both ends of the edge lie on different supernodes then another checking is done to update the weight of the supernodes. If the weights of the supernodes where both ends of the edge belong is greater than the weight of edge, then the weights of the supernodes are updated to the edge weight. After the implementation till Line 33 the supernodes weights will contain the cheapest edge weight by which each supernode connects another supernode. Traversing each subset and connecting to another subset by its cheapest edge all the subsets become connected (Line 34-36).

III. SIMULATION

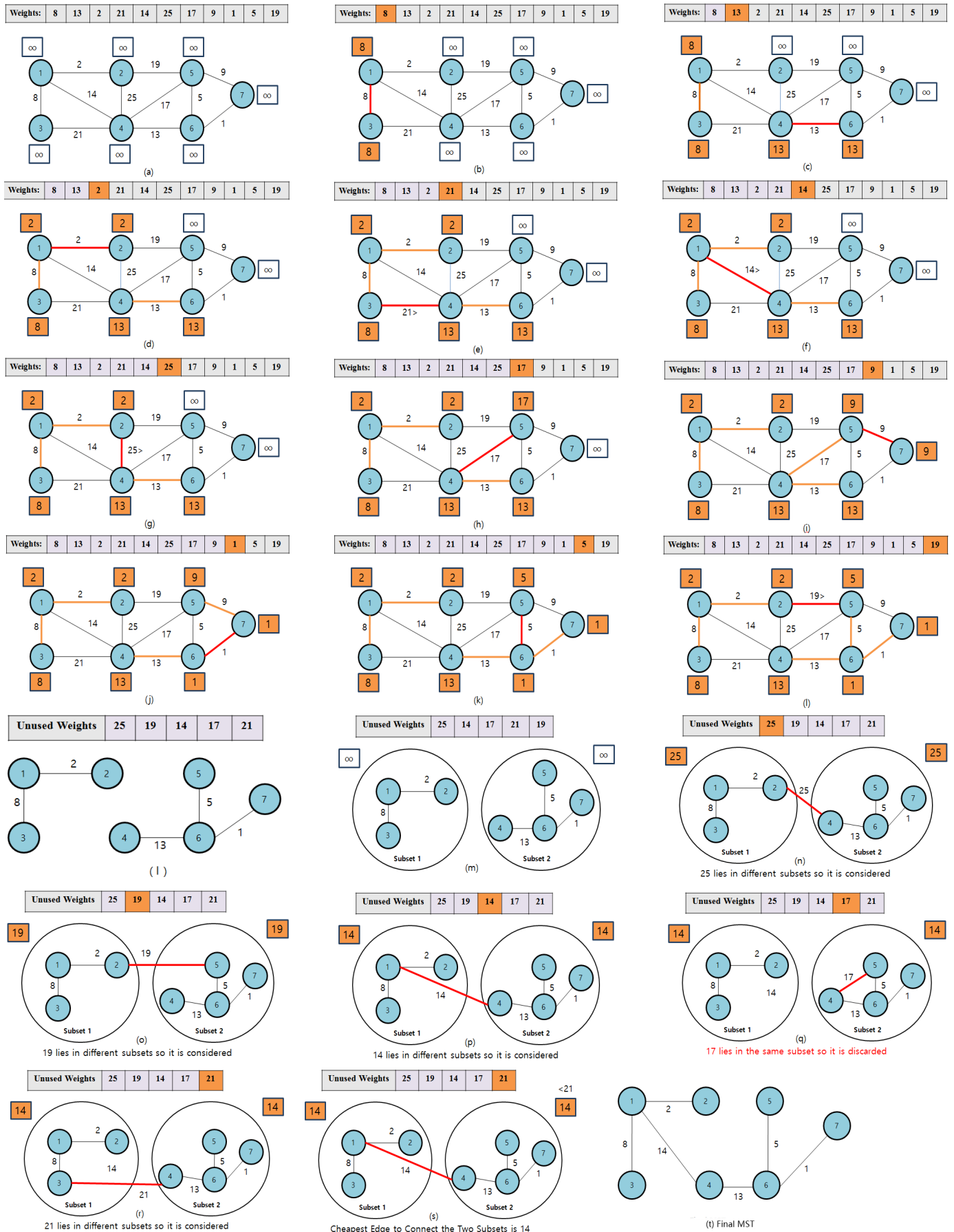


Fig. 1 Simulation of the algorithm

IV. TIME COMPLEXITY ANALYSIS OF THE PROPOSED ALGORITHM

In this section, the complexities of the algorithm at the best case and worst case is discussed.

- **Line (1-3)** iterates V times and so the complexity is $O(|V|)$ to initialize infinity to all the vertices.
- **Line (4-13)** runs in E times. Line (7-9) and line (10-12) has $O(1)$ complexity for each execution. So, for all the edges, it makes the total complexity for line (4-13) $O(|E|)$.
- **Line (14-16)** has complexity $O(|V|)$ since each of the vertex is traversed and its corresponding cheapest edge is selected for making the subsets.
- Similarly, **line (17-19)** needs $O(|V|)$ time for initializing infinity to the subsets.
- **Line (20-33)** iterates E' times where E' is the set of edges not used while formation of subsets. Line 25 has complexity $O(\log(|E'|))$ for each find-set operation. Therefore, the total time complexity for line (20-33) is $O(\log |E'|) * O(|E'|) \approx O(|E'| \log |E'|)$.
- **Line (34-36)** runs in $O(S) \approx O(|V|)$ times.

So, the total time complexity of the algorithm is $O(|V|) + O(|E|) + O(|E'| \log |E'|) \approx O(|E| \log |E|)$. At worst case i.e. for most dense graph $E = V^2$. So, the worst case complexity is $O(|E| \log |V^2|) \approx O(2 * |E| \log |V|) \approx O(|E| \log |V|)$. In sparse graphs the number of edges and the number of vertices are almost equal ($E = V$). So, the time complexity of the algorithm for best case is $O(|V| \log |V|)$

V. PROOF OF CORRECTNESS

Theorem: The proposed algorithm constructs a minimum-weight spanning tree when applied to a connected graph.

Lemma 1: The subsets formed does not contain any cycles within them.

Let three vertices u , v and z be considered in the same subset. With the contradiction method we will prove that there exists no cycle among these three vertices. Let us assume that uv is the cheapest edge for the vertex or node u by which it connects another node and thus the node weight of u is $W(uv)$. Now if uv is not the cheapest edge for the node v then let vz is the cheapest edge for it. So $W(vz) < W(uv)$ and weight of node v is $W(vz)$. Then again if the cheapest edge for z is vz then weight of the node z will be $W(vz)$ but if that is not the case then let zu be the cheapest edge for z . Therefore, $W(zu) < W(vz)$. Now

$$W(vz) < W(uv) \quad (1)$$

$$W(zu) < W(vz) \quad (2)$$

$$\text{So } W(zu) < W(uv) \quad (3)$$

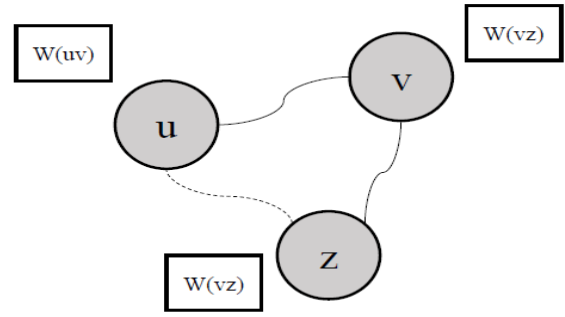


Fig. 2

So, $W(zu) < W(uv)$ which makes zu the cheapest edge for the node u but that is not possible as it contradicts the statement uv is the cheapest edge for u . Therefore, it is proved that no cycle exists among the vertices in the same subset.

Lemma 2: Joining of the subsets forms a spanning tree.

Joining of subsets is the last phase of the algorithm. We prove that after connecting all the subsets, the graph obtained is a tree. Each time when we choose an edge to connect two subsets it is checked whether both end nodes of the edge reside in different subsets. Cycles can only be formed during subset joining if any edge is considered whose end nodes belong to the same subset. Since no such edge is ever chosen so it can be said that there is no chance of cycle formation.

After the joining of subsets are completed if still there are any more component not joined then it indicates that there is no edge joining it and the graph G is not connected but that is not possible because the graph G is connected. Therefore, the final graph formed is acyclic and all the vertices are connected which makes it a spanning tree

Lemma 3: The spanning tree formed by joining of subsets is of minimum weight.

Let T be the resulting tree generated by the algorithm and T^* be the minimum spanning tree of the connected graph G . We have to prove that $T = T^*$. We assume, $T \neq T^*$

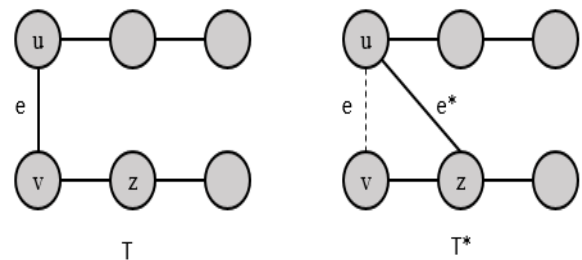


Fig. 3

$$\text{So, } W(T^*) \leq W(T) \quad (4)$$

In the tree T there exists an edge e which does not belong to T^* . To compensate for the edge e , edge e^* exists in T^* . Now in T edge e is chosen to connect node u because it is the cheapest edge for u to be connected to another node. If the cost or weight of edge e^* was less than or equal to that of e then e^* would have been the cheapest edge of u to be connected to another node. Thus,

$$W(e) \leq W(e^*) \quad (5)$$

Thus we can add edge e in T^* and remove e^* since $W(e) < W(e^*)$. Now

$$W(T^*) - W(e^*) + W(e) \leq W(T^*) \quad (6)$$

The inequality (1) will be true if and only if $W(e) \leq W(e^*)$ which is true as well. Let us consider $W(T^*) = 30$, $W(e) = 5$ and $W(e^*) = 8$. Using this values in (6) we get,

$$27 \leq 30 \text{ which can never be true} \quad (7)$$

Thus, we see that the weight of T^* decreases when we use edge e instead of e^* which clearly indicates T^* cannot be the minimum spanning tree for the graph G which contradicts our statement T^* is the minimum spanning tree. It is seen that when T^* is made similar to T by using edge e , not edge e^* then $W(T^*)$ becomes minimum. As a result, $T^* = T$. Therefore, T is the minimum spanning tree for the graph G .

VI. DISCUSSION AND CONCLUSION

In this paper, we have proposed a new approach to find MST algorithm by reducing the number of edges using set and disjoint algorithm. As in this method edge sorting has not been performed, number of edges are reduced and connects subsets without cycles in the first step, time complexity of this algorithm is quite convenient. It is mentionable that, the proposed algorithm has the same time complexity as that of the most commonly used algorithms like Boruvka's Algorithm, Prim's Algorithm and Kruskal's Algorithm which is $O(E \log V)$. However the proposed algorithm could not match the complexity of Filter-Kruskal Algorithm i.e. $O(E + V \log(V \log(E/V)))$ but still it can be used as a different approach. We observed that our proposed algorithm works on less dense graph efficiently and hope that further research will improve the complexity of this algorithm for dense graph with better performance.

REFERENCES

- [1] M. Held and R. M. Karp, "The traveling-salesman problem and minimum spanning trees," *Operations Research*, vol. 18, no. 6, pp. 1138–1162, 1970.
- [2] K. Bharath-Kumar and J. Jaffe, "Routing to multiple destinations in computer networks," *IEEE Transactions on communications*, vol. 31, no. 3, pp. 343–351, 1983.
- [3] L. An, Q.-S. Xiang, and S. Chavez, "A fast implementation of the minimum spanning tree method for phase unwrapping," *IEEE transactions on medical imaging*, vol. 19, no. 8, pp. 805–808, 2000.
- [4] N. Akpan and I. Iwok, "A minimum spanning tree approach of solving a transportation problem," *International Journal of Mathematics and Statistics Invention*, vol. 5, no. 3, pp. 09–18, 2017.
- [5] R. L. Graham and P. Hell, "On the history of the minimum spanning tree problem," *Annals of the History of Computing*, vol. 7, no. 1, pp. 43–57, 1985.
- [6] R. C. Prim, "Shortest connection networks and some generalizations," *Bell Labs Technical Journal*, vol. 36, no. 6, pp. 1389–1401, 1957.
- [7] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical society*, vol. 7, no. 1, pp. 48–50, 1956.
- [8] Reverse Delete Algorithm, url = https://en.wikipedia.org/wiki/Reverse-delete_algorithm, urldate = 2018–08–17.
- [9] D. R. Karger, P. N. Klein, and R. E. Tarjan, "A randomized linear-time algorithm to find minimum spanning trees," *Journal of the ACM (JACM)*, vol. 42, no. 2, pp. 321–328, 1995.
- [10] P. Biswas, M. Goel, H. Negi, and M. Datta, "An efficient greedy minimum spanning tree algorithm based on vertex associative cycle detection method," *Procedia Computer Science*, vol. 92, pp. 513–519, 2016.
- [11] M. L. Fredman and D. E. Willard, "Trans-dichotomous algorithms for minimum spanning trees and shortest paths," *Journal of Computer and System Sciences*, vol. 48, no. 3, pp. 533–551, 1994.
- [12] V. Osipov, P. Sanders, and J. Singler, "The filter-kruskal minimum spanning tree algorithm," in *2009 Proceedings of the Eleventh Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 2009, pp. 52–61.
- [13] C. Zhong, M. Malinen, D. Miao, and P. Fränti, "A fast minimum spanning tree algorithm based on k-means," *Information Sciences*, vol. 295, pp. 1–17, 2015.
- [14] S. Saxena, P. Verma, and D. S. Rajpoot, "Clustering based minimum spanning tree algorithm," in *Contemporary Computing (IC3), 2017 Tenth International Conference on*. IEEE, 2017, pp. 1–3.
- [15] V. Lončar, S. Škrbić, and A. Balaž, "Parallelization of minimum spanning tree algorithms using distributed memory architectures," in *Transactions on Engineering Technologies*. Springer, 2014, pp. 543–554.
- [16] D. A. Bader and G. Cong, "Fast shared-memory algorithms for computing the minimum spanning forest of sparse graphs," in *Parallel and Distributed Processing Symposium, 2004. Proceedings. 18th International*. IEEE, 2004, p. 39.